Assignment 0: Code Walkthrough Answers With Details

Question 1: What is the VM system called that is configured for assignment 0?

Answer:
dumbvm.

```
# For the early assignments, we supply a very stupid MIPS-only skeleton
# of a VM system. It is just barely capable of running a single userlevel
# program as long as that program's not very large.
defoption   dumbvm
machine mips optfile dumbvm    arch/mips/vm/dumbvm.c
```

Where to find it:

- os161-1.99/kern/arch/mips/conf/conf.arch: defines the dumbvm

Question 2: Which register number is used for the stack pointer (sp) in OS/161?

Answer:
29.

```
67   #define k1  $27    /* kernel temporary 1 */
68   #define gp  $28    /* global pointer */
69   #define sp  $29    /* stack pointer */
70   #define s8  $30    /* saved (callee-save) 8 = frame pointer */
```

Where to find it:
- os161-1.99/kern/arch/mips/include/kern/regdefs.h: defines "sp" as "$29".

Question 3: What bus/busses does OS/161 support?

Answer:
LAMEbus.

```
/*
 * Generic bus interface file.
 *
 * The only bus on System/161 is LAMEbus.
 * This would need to be a bit more complicated if that weren't the case.
 */
```

Where to find it:

- os161-1.99/kern/arch/sys161/include/bus.h: says the only bus on System/161 is LAMEbus.

Question 4: Why do we use typedefs like uint32_t instead of simply saying "int"?

Answer:
uint32_t specify exact size. A plain int can vary between platforms and compilers.

```
typedef __u8 uint8_t;
typedef __u16 uint16_t;
typedef __u32 uint32_t;
typedef __u64 uint64_t;
typedef __size_t size_t;
```

Where to find it:
- os161-1.99/kern/include/types.h: maps uint32_t to __u32.

Question 5: What function is called when user-level code generates a fatal fault?

Answer:
kill_curthread() is called.

```
/*
 * Function called when user-level code hits a fatal fault.
 */
static
void
kill_curthread(vaddr_t epc, unsigned code, vaddr_t vaddr)
{
    int sig = 0;
```

Where to find it:
- os161-1.99/kern/arch/mips/locore/trap.c: defines kill_curthread().

Question 6: How frequently are hardclock interrupts generated?

Answer:

```
/* Make synchronization more exciting :) */
#define HZ  10000
#else
/* More realistic value */
#define HZ  100
```

Where to find it:
- os161-1.99/kern/include/clock.h:

What it does:
hardclock() updates CPU clock counters and periodically triggers scheduling and

thread migration checks. In this tree it also calls thread_yield().

Question 7: How many characters are allowed in an SFS volume name?

Answer:
SFS has a 32-byte volume-name field.

```
#define SFS_BLOCKSIZE     512          /* size of our blocks */
#define SFS_VOLNAME_SIZE  32           /* max length of volume name */
#define SFS_NDIRECT       15           /* # of direct blocks in inode */
#define SFS_DBPERIDB      128          /* # direct blks per indirect blk */
#define SFS_NAMELEN       60           /* max length of filename */
#define SFS_SB_LOCATION    0           /* block the superblock lives in */
```

Where to find it:
- os161-1.99/kern/include/kern/sfs.h: defines SFS_VOLNAME_SIZE as 32 and uses it for sp_volname.

Question 8: How many direct blocks does an SFS file have?

Answer:
An SFS inode has 15 direct blocks.

```
#define SFS_BLOCKSIZE     512          /* size of our blocks */
#define SFS_VOLNAME_SIZE  32           /* max length of volume name */
#define SFS_NDIRECT       15           /* # of direct blocks in inode */
#define SFS_DBPERIDB      128          /* # direct blks per indirect blk */
#define SFS_NAMELEN       60           /* max length of filename */
#define SFS_SB_LOCATION    0           /* block the superblock lives in */
```

Where to find it:
- os161-1.99/kern/include/kern/sfs.h: defines SFS_NDIRECT as 15.

Question 9: What is the standard interface to a file system?

Answer:
The standard filesystem interface is struct fs. A filesystem must provide fs_sync, fs_getvolname, fs_getroot, and fs_unmount.

```
/*
 * Abstract filesystem. (Or device accessible as a file.)
 *
 * Operations:
 *
 *      fs_sync       - Flush all dirty buffers to disk.
 *      fs_getvolname - Return volume name of filesystem.
 *      fs_getroot    - Return root vnode of filesystem.
 *      fs_unmount    - Attempt unmount of filesystem.
 *
```

Where to find it:
- os161-1.99/kern/include/fs.h: defines struct fs and the FSOP_* macros.

Question 10: What function puts a thread to sleep?

Answer:
wchan_sleep() puts the current thread to sleep on a wait channel.

```
/*
 * Go to sleep on a wait channel. The current thread is suspended
 * until awakened by someone else, at which point this function
 * returns.
 *
 * The channel must be locked, and will have been *unlocked* upon
 * return.
 */
void wchan_sleep(struct wchan *wc);
```

Where to find it:
- os161-1.99/kern/include/wchan.h: declares wchan_sleep().

Question 11: How large are OS/161 pids?

Answer:
pid_t is a signed 32-bit type.

```
typedef __u32 __mode_t;      /* File access mode */
typedef __u16 __nlink_t;     /* Number of links (intentionally
typedef __i64 __off_t;       /* Offset within file */
typedef __i32 __pid_t;       /* Process ID */
typedef __u64 __rlim_t;      /* Resource limit quantity */
typedef __u8 __sa_family_t;/* Socket address family */
typedef __i64 __time_t;      /* Time in seconds */
```

Where to find it:
- os161-1.99/kern/include/kern/types.h: typedefs __pid_t as __i32.

Question 12: What operations can you do on a vnode?

Answer:
Vnode operations include open, close, reclaim, read, readlink, getdirentry, write, ioctl, stat, gettype, tryseek, fsync, mmap, truncate, namefile, creat, symlink, mkdir, link, remove, rmdir, rename, lookup, and lookparent.

```
* Abstract operations on a vnode.
*
* These are used in the form VOP_FOO(vnode, args), which are macros
* that expands to vnode->vn_ops->vop_foo(vnode, args). The operations
* "foo" are:
*
*    vop_open        - Called on *each* open() of a file. Can be used to
*                      reject illegal or undesired open modes. Note that
*                      various operations can be performed without the
*                      file actually being opened.
*                      The vnode need not look at O_CREAT, O_EXCL, or
*                      O_TRUNC, as these are handled in the VFS layer.
*
*                      VOP_OPEN should not be called directly from above
*                      the VFS layer - use vfs_open() to open vnodes.
*                      This maintains the open count so VOP_CLOSE can be
*                      called at the right time.
*
*    vop_close       - To be called on *last* close() of a file.
*
*                      VOP_CLOSE should not be called directly from above
*                      the VFS layer - use vfs_close() to close vnodes
*                      opened with vfs_open().
*
*    vop_reclaim     - Called when vnode is no longer in use. Note that
*                      this may be substantially after vop_close is
*                      called.
```

Where to find it:
- os161-1.99/kern/include/vnode.h: defines struct vnode_ops and VOP_* macros.

Question 13: What is the maximum path length in OS/161?

Answer:
The maximum path length is PATH_MAX = 1024.

```
/* Longest full path name */
#define __PATH_MAX      1024
```

Where to find it:
- os161-1.99/kern/include/kern/limits.h: defines __PATH_MAX as 1024.

Question 14: What is the system call number for a reboot?

Answer:
119.

```
#define SYS_sync          118
#define SYS_reboot        119
//#define SYS___sysctl    120
```

Where to find it:
- os161-1.99/kern/include/kern/syscall.h: defines SYS_reboot as 119.

Question 15: Where is STDIN_FILENO defined?

Answer:
STDIN_FILENO is defined as 0 in kern/include/kern/unistd.h.

```
/* Constants for read/write/etc: special file handles */
#define STDIN_FILENO  0      /* Standard input */
#define STDOUT_FILENO 1      /* Standard output */
#define STDERR_FILENO 2      /* Standard error */
```

Where to find it:
- os161-1.99/kern/include/kern/unistd.h: defines STDIN_FILENO, STDOUT_FILENO, and STDERR_FILENO.

Question 16: What does kmain() do?

Answer:
kmain() calls boot() to initialize the kernel, then calls menu(arguments).

```
/*
 * Kernel main. Boot up, then fork the menu thread; wait for a reboot
 * request, and then shut down.
 */
void
kmain(char *arguments)
{
    boot();

    menu(arguments);

    /* Should not get here */
}
```

Where to find it:
- os161-1.99/kern/startup/main.c: defines boot(), shutdown(), sys_reboot(), and kmain().

Question 17: What is the difference between splhigh and spl0?

Answer:
splhigh() disables interrupts on the current CPU by setting IPL_HIGH. spl0() enables interrupts by setting IPL_NONE.

```
spl0()        sets IPL to 0, enabling all interrupts.
splhigh()     sets IPL to the highest value, disabling all interrupts.
```

Where to find it:
- os161-1.99/kern/include/spl.h: documents spl0(), splhigh(), and splx().

Question 18: What does splx return?

Answer:
splx() returns the old interrupt priority level before changing to the requested level.

```
     spl0()        sets IPL to 0, enabling all interrupts.
     splhigh()     sets IPL to the highest value, disabling all interrupts.
     splx(s)       sets IPL to S, enabling whatever state S represents.

All three return the old interrupt state. Thus, these are commonly used
as follows:

     int s = splhigh();
     [ code ]
     splx(s);

Note that these functions only affect interrupts on the current
processor.
```

Where to find it:
- os161-1.99/kern/include/spl.h: states that spl0(), splhigh(), and splx() all return the old interrupt state.

Question 19: What is a zombie?

Answer:
A zombie is a thread that has exited but has not yet been destroyed.

```
/* States a thread can be in. */
typedef enum {
    S_RUN,       /* running */
    S_READY,     /* ready to run */
    S_SLEEP,     /* sleeping */
    S_ZOMBIE,    /* zombie; exited but not yet deleted */
} threadstate_t;
```

Where to find it:
- os161-1.99/kern/include/thread.h: defines S_ZOMBIE as "zombie; exited but not

yet deleted".

Question 20: What does a device name in OS/161 look like?

Answer:
A device name in pathname syntax looks like "devname:"

```
vfs_adddev    - Add a device to the VFS named device list. If
                MOUNTABLE is zero, the device will be accessible
                as "DEVNAME:". If the mountable flag is set, the
                device will be accessible as "DEVNAMEraw:" and
                mountable under the name "DEVNAME". Thus, the
                console, added with MOUNTABLE not set, would be
                accessed by pathname as "con:", and lhd0, added
                with mountable set, would be accessed by
                pathname as "lhd0raw:" and mounted by passing
                "lhd0" to vfs_mount.
```

Where to find it:
- os161-1.99/kern/include/vfs.h: documents that devices are accessible as "DEVNAME:".

Question 21: What does a raw device name in OS/161 look like?

Answer:
A raw device name appends "raw" before the colon, for example "lhd0raw:".

```
* kd_rawname - Name of raw device (eg, "lhd0raw"). Is non-NULL if and
*               only if this device can have a filesystem mounted on
*               it.
*
```

Where to find it:
- os161-1.99/kern/vfs/vfslist.c: comments give "lhd0raw" as the raw device name

Question 22: What lock protects the vnode reference count?

Answer:
global VFS big lock, vfs_biglock.

```c
/*
 * Increment refcount.
 * Called by VOP_INCREF.
 */
void
vnode_incref(struct vnode *vn)
{
    KASSERT(vn != NULL);

    vfs_biglock_acquire();

    vn->vn_refcount++;

    vfs_biglock_release();
}
```

Where to find it:
- os161-1.99/kern/vfs/vnode.c: vnode_incref() and vnode_decref() acquire vfs_biglock before changing vn_refcount.

Question 23: What device types are currently supported?

Answer:
The VFS device layer currently distinguishes block devices and character devices.

```c
/*
 * Return the type. A device is a "block device" if it has a known
 * length. A device that generates data in a stream is a "character
 * device".
 */
static
int
dev_gettype(struct vnode *v, mode_t *ret)
{
    struct device *d = v->vn_data;
    if (d->d_blocks > 0) {
        *ret = S_IFBLK;
    }
    else {
        *ret = S_IFCHR;
    }
    return 0;
}
```

Where to find it:

- os161-1.99/kern/vfs/device.c: dev_gettype() returns S_IFBLK when d_blocks > 0, otherwise S_IFCHR.